



**UNIVERSIDADE FEDERAL DO TOCANTINS
CÂMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE ENGENHARIA ELÉTRICA**

MAURÍCIO MOURA DE CARVALHO

RELATÓRIO 5 DE PDS: TRANSFORMADA DE FOURIER DISCRETA

PALMAS (TO)

2026

Maurício Moura de Carvalho

Relatório 5 de PDS: Transformada de Fourier Discreta

Relatório de Laboratório apresentado à disciplina de Processamento Digital de Sinais do curso de Engenharia Elétrica da Universidade Federal do Tocantins.

Professor: Dr. Ivan Ney Alvizuri Romani

PALMAS (TO)

2026

SUMÁRIO

1	INTRODUÇÃO	1
2	METODOLOGIA E PROCEDIMENTOS PRÁTICOS	2
2.1	Parte I – Equações Vetoriais de Análise e Síntese da TFD	2
2.2	Parte II – TFTD e TFD de 4 Pontos do Pulso Retangular	2
2.3	Parte III – Alta Densidade vs. Alta Resolução Espectral	2
2.4	Parte IV – Tempo de Execução da FFT	2
3	RESULTADOS E DISCUSSÕES	4
3.1	Parte I: Validação da Matriz W_N	4
3.2	Parte II: Pulso Retangular – TFTD vs. TFD de 4 Pontos	5
3.3	Parte III: Alta Densidade vs. Alta Resolução Espectral	8
3.4	Parte IV: Tempo de Execução da FFT	11
4	CONCLUSÃO	14
	REFERÊNCIAS	15

1 INTRODUÇÃO

Este relatório apresenta os resultados do quinto laboratório da disciplina de Processamento Digital de Sinais (PDS), dedicado ao tema da **Transformada de Fourier Discreta (TFD)**. Diferentemente da Transformada de Fourier em Tempo Discreto (TFTD), que produz um espectro contínuo e periódico em frequência, a TFD opera sobre um número finito de amostras tanto no domínio do tempo quanto no domínio da frequência, sendo a ferramenta fundamental para o processamento digital de sinais em sistemas computacionais reais, onde apenas um número finito de pontos de frequência pode ser representado.

A TFD pode ser entendida como o resultado da amostragem da TFTD em N pontos equidistantes no intervalo $[0, 2\pi)$, e sua formulação matricial – por meio da matriz W_N – evidencia a relação direta entre as equações de análise e síntese. Esta dualidade matricial é também a base teórica do algoritmo da Transformada Rápida de Fourier (FFT), que reduz a complexidade computacional do cálculo da TFD de $O(N^2)$ para $O(N \log N)$ ao explorar a fatoração de N .

O objetivo desta prática foi: (i) implementar e validar as equações vetoriais de análise e síntese da TFD; (ii) comparar a TFTD com a TFD de 4 pontos de um pulso retangular, evidenciando a relação de amostragem entre ambas; (iii) investigar a distinção entre densidade espectral e resolução espectral por meio da separação de duas senoides de frequências próximas; e (iv) avaliar experimentalmente o tempo de execução da FFT em função de N , contrastando o desempenho para valores primos e compostos. As implementações foram realizadas no ambiente Octave (EATON et al., 2025), seguindo o roteiro de laboratório (UFT, 2026) e com suporte teórico nas referências de Proakis e Manolakis (PROAKIS; MANOLAKIS, 2021), Oppenheim e Willsky (OPPENHEIM; WILLISKY; NAWAB, 1996) e Ingle e Proakis (INGLE; PROAKIS, 2017).

2 METODOLOGIA E PROCEDIMENTOS PRÁTICOS

2.1 Parte I – Equações Vetoriais de Análise e Síntese da TFD

As equações de análise e síntese da TFD podem ser escritas em forma vetorial por meio da matriz W_N , cujos elementos são as raízes N -ésimas da unidade:

$$W_N(k, n) = e^{-j\frac{2\pi}{N}kn}, \quad 0 \leq k, n \leq N-1 \quad (1)$$

de modo que a análise (equação a) e a síntese (equação b) tornam-se, respectivamente:

$$\mathbf{X} = W_N \mathbf{x}, \quad \mathbf{x} = \frac{1}{N} W_N^* \mathbf{X} \quad (2)$$

em que $\mathbf{x}$ e $\mathbf{X}$ são vetores-coluna contendo, respectivamente, as N amostras de $x[n]$ e os N coeficientes $X[k]$. Foi implementado um programa no Octave para construir explicitamente a matriz W_N , calcular $\mathbf{X}$ a partir de $\mathbf{x}$ pela equação (a), reconstruir $\mathbf{x}$ a partir de $\mathbf{X}$ pela equação (b), e comparar os resultados com a função nativa `fft` do Octave.

2.2 Parte II – TFTD e TFD de 4 Pontos do Pulso Retangular

Para a sequência $x[n] = 1, 0 \leq n \leq 3$ (e 0 para os demais n), foram calculadas: (a) a TFTD $X(e^{j\omega})$, avaliada pela definição em um conjunto denso de frequências, com plotagem de magnitude e fase; e (b) a TFD de 4 pontos de $x[n]$, com plotagem de magnitude e fase sobrepostas à TFTD, permitindo verificar visualmente que a TFD corresponde exatamente às amostras da TFTD nas frequências discretas $\omega_k = 2\pi k/N$.

2.3 Parte III – Alta Densidade vs. Alta Resolução Espectral

Para a sequência $x[n] = \cos(0,48\pi n) + \cos(0,52\pi n)$, soma de duas senoides de frequências normalizadas muito próximas ($f_1 = 0,24$ e $f_2 = 0,26$ ciclos/amostra), foram calculadas: (a) a TFD considerando $0 \leq n \leq 10$ ($N = 11$ amostras); e (b) a TFD considerando $0 \leq n \leq 100$ ($N = 101$ amostras). O objetivo foi investigar se as duas componentes de frequência podem ser distinguidas (resolvidas) em cada caso, ilustrando a diferença conceitual entre uma TFD de *alta densidade* (mais pontos de frequência, mesma duração do sinal) e uma TFD de *alta resolução* (mais amostras do sinal, portanto maior duração de observação).

2.4 Parte IV – Tempo de Execução da FFT

Foi avaliado o tempo de execução da função `fft` do Octave para todo N inteiro no intervalo $2 \leq N \leq 2048$, utilizando as funções `clock` e `etime` para medir o tempo de CPU

decorrido, conforme solicitado no roteiro. Para reduzir a influência de ruído de medição, cada FFT foi repetida múltiplas vezes para cada valor de N e o tempo médio foi calculado. Os valores de N foram classificados em primos e compostos (com auxílio da função `isprime`) para investigar o impacto da fatorabilidade de N no desempenho do algoritmo.

3 RESULTADOS E DISCUSSÕES

3.1 Parte I: Validação da Matriz W_N

O Código 1 implementa a matriz W_N definida pela equação (1) para $N = 4$, utilizando a sequência de teste $x[n] = \{1, 2, 3, 4\}$.

```
1  % Parte I - Equacoes da TFD em forma vetorial (matriz WN)
2  % Analise: X = WN*x
3  % Sintese: x = (1/N)*WN'*X, com WN' = conj(WN), pois WN e
   simetrica
4
5  clear; clc; close all;
6
7  N = 4;
8  n = 0:N-1;
9  k = n;
10 WN = exp(-1j*2*pi*k'*n/N); % WN(k+1,n+1) = exp(-j*2*pi*k*n/
   N)
11
12 x = [1 2 3 4]; % sequencia de teste
13
14 X = WN*x.'; % Analise (equacao a)
15 x_rec = (1/N)*conj(WN)*X; % Sintese (equacao b)
16
17 disp('Matriz WN (N=4):'); disp(WN);
18 disp('Sequencia original x:'); disp(x);
19 disp('TFD via X = WN*x:'); disp(X. ');
20 disp('TFD via fft (Octave), para comparacao:'); disp(fft(x));
21 disp('Erro entre WN*x e fft(x) (norma):'); disp(norm(X.' -
   fft(x)));
22 disp('Sequencia reconstruida via x = (1/N)*WN_conj*X:'); disp
   (x_rec. ');
23 disp('Erro de reconstrucao (norma):'); disp(norm(x.' - x_rec
   ));
```

Código 3.1 – Script para a Parte I.

A construção de W_N é feita de forma totalmente vetorizada, com $k' * n$ gerando a matriz de produtos externos kn e o operador $\exp$ aplicado elemento a elemento. Para $N = 4$, $W_N(k, n) =$

$e^{-j\frac{\pi}{2}kn} = (-j)^{kn}$, o que produz a matriz:

$$W_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

Simetria de W_N . Observa-se que $W_N(k, n) = W_N(n, k)$, pois o produto kn é simétrico em k e n – ou seja, a matriz W_N é simétrica ($W_N^T = W_N$), embora não seja Hermitiana. Essa propriedade tem uma consequência prática direta na equação de síntese: como W_N é simétrica, $(W_N^*)^T = W_N^*$, de modo que a transposição pode ser omitida na implementação, bastando calcular $\text{conj}(WN) * X$ em vez de $\text{conj}(WN) . ' * X$, exatamente como feito no Código 1 (linha 15).

Ortogonalidade e fator de escala. As colunas de W_N são ortogonais entre si e cada uma possui norma² igual a N , de modo que $W_N^H W_N = N I_N$ (em que W_N^H é a conjugada transposta). Isso implica que $W_N / \sqrt{N}$ é uma matriz unitária, e justifica diretamente por que a equação de síntese requer o fator de normalização $1/N$: aplicar W_N^* (que para W_N simétrica equivale a W_N^H) sobre $\mathbf{X} = W_N \mathbf{x}$ resulta em $W_N^* W_N \mathbf{x} = N \mathbf{x}$, sendo necessário dividir por N para recuperar $\mathbf{x}$.

Verificação numérica. A execução do Código 1 produziu $X = W_4 \mathbf{x} = \{10, -2 + 2j, -2, -2 - 2j\}$, valor idêntico ao retornado pela função nativa `fft(x)` do Octave, com erro de norma da ordem de $2,3 \times 10^{-15}$ – compatível com a precisão de ponto flutuante de double precisão e não com qualquer divergência conceitual entre as duas formulações. A reconstrução $\mathbf{x}_{rec} = \frac{1}{N} W_4^* \mathbf{X}$ recuperou a sequência original $\{1, 2, 3, 4\}$ com erro de norma da ordem de $1,0 \times 10^{-15}$, confirmando experimentalmente que as equações de análise (2)a e síntese (2)b formam, de fato, um par exato de transformação direta e inversa.

3.2 Parte II: Pulso Retangular – TFTD vs. TFD de 4 Pontos

O Código 2 calcula a TFTD do pulso retangular $x[n] = 1, 0 \leq n \leq 3$, pela definição, e compara com a TFD de 4 pontos obtida via `fft`.

```

1  % Parte II - x[n] = 1, 0 <= n <= 3
2  % a) TFTD X(e^jw) calculada pela definicao (espectro continuo
   )
3  % b) TFD de 4 pontos, comparada com as amostras da TFTD em w
   = 2*pi*k/N
4
5  clear; clc; close all;
6  set(0, 'defaultaxesfontname', 'Arial');
7  set(0, 'defaultaxesfontsize', 11);
8

```

```

9  n = 0:3;
10 x = ones(1,4);
11 N = 4;
12
13 % a) TFTD continua
14 w = linspace(-pi, pi, 1001);
15 Xw = zeros(1, length(w));
16 for idx = 1:length(w)
17     Xw(idx) = sum(x .* exp(-1j*w(idx)*n));
18 end
19
20 % b) TFD de N=4 pontos
21 Xk = fft(x, N);
22 k = 0:N-1;
23 wk = 2*pi*k/N;
24 wk_plot = wk; wk_plot(wk_plot > pi) = wk_plot(wk_plot > pi) -
    2*pi; % p/ exibir em [-pi,pi]
25
26 figure('Position', [100 100 900 400]);
27
28 subplot(1,2,1);
29 plot(w/pi, abs(Xw), 'b', 'LineWidth', 1.5); hold on;
30 stem(wk_plot/pi, abs(Xk), 'r', 'filled');
31 xlabel('\omega/\pi'); ylabel('|X(e^{j\omega})|, |X[k]|');
32 title('Magnitude: TFTD (linha) e TFD N=4 (marcadores)');
33 legend('TFTD', 'TFD', 'Location', 'south');
34 grid on;
35
36 subplot(1,2,2);
37 plot(w/pi, angle(Xw), 'b', 'LineWidth', 1.5); hold on;
38 stem(wk_plot/pi, angle(Xk), 'r', 'filled');
39 xlabel('\omega/\pi'); ylabel('\angle X(e^{j\omega}), \angle X
    [k] (rad)');
40 title('Fase: TFTD (linha) e TFD N=4 (marcadores)');
41 legend('TFTD', 'TFD', 'Location', 'south');
42 grid on;
43
44 drawnow;
45 frame = getframe(gcf);
46 imwrite(frame.cdata, '../figuras/parte2.png');
47

```

```

48 disp('TFTD de 4 pontos X[k]:'); disp(Xk);
49 disp('Parte II concluída. Figura salva.');
```

Código 3.2 – Script para a Parte II.

Para um pulso retangular de comprimento $N = 4$, a forma fechada da TFTD é o núcleo de Dirichlet:

$$X(e^{j\omega}) = \sum_{n=0}^3 e^{-j\omega n} = e^{-j\frac{3\omega}{2}} \frac{\sin(2\omega)}{\sin(\omega/2)} \quad (3)$$

cuja magnitude $|X(e^{j\omega})| = |\sin(2\omega)/\sin(\omega/2)|$ é plotada na Figura 1 (linha azul), juntamente com os 4 pontos da TFD $X[k]$ (marcadores vermelhos), calculados em $\omega_k = 2\pi k/4$, $k = 0, 1, 2, 3$, e remapeados para o intervalo $[-\pi, \pi]$ por conveniência de visualização.

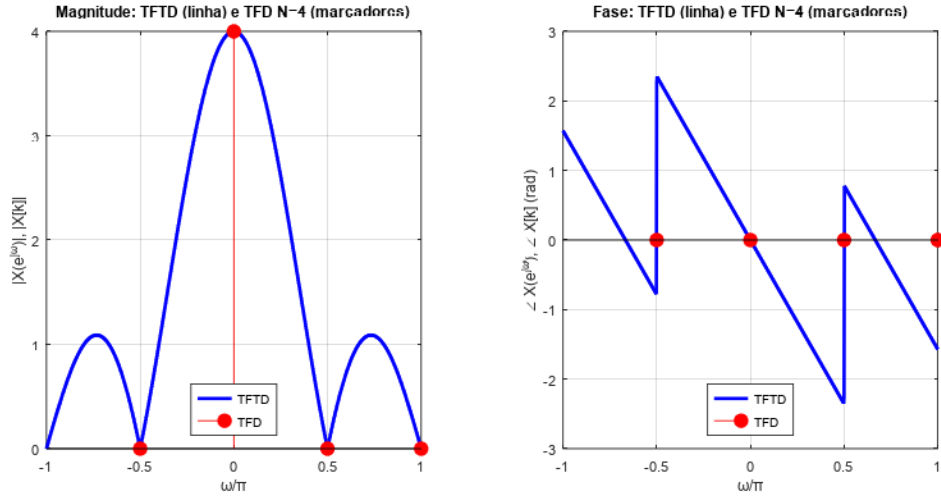


Figura 1 – TFTD (linha azul) e TFD de 4 pontos (marcadores vermelhos) do pulso retangular $x[n] = 1, 0 \leq n \leq 3$: magnitude e fase.

Amostragem da TFTD pela TFD. A Figura 1 evidencia, de forma direta, a propriedade central que relaciona as duas transformadas: *a TFD de N pontos de uma sequência de duração finita N é exatamente a TFTD dessa sequência amostrada nas N frequências discretas $\omega_k = 2\pi k/N$* . Todos os 4 marcadores vermelhos, tanto na magnitude quanto na fase, caem exatamente sobre a curva contínua azul, sem qualquer ajuste ou interpolação – confirmação visual direta da relação $X[k] = X(e^{j\omega})|_{\omega=2\pi k/N}$.

Resultado numérico peculiar. A TFD calculada foi $X[k] = \{4, 0, 0, 0\}$. Esse resultado, embora pareça trivial, ilustra um caso particularmente didático: como a janela de observação ($N = 4$ amostras) coincide exatamente com o suporte do pulso retangular, o sinal observado é constante ($x[n] = 1$ para todo n na janela), de modo que toda a sua energia se concentra na componente DC ($k = 0$, valendo $4 = \sum_n x[n]$). Os zeros do núcleo de Dirichlet (3) ocorrem precisamente em $\omega = \pi/2, \pi, 3\pi/2$ – que são exatamente as frequências discretas amostradas pela TFD para $k = 1, 2, 3$. Esta coincidência não é acidental: sempre que um sinal periódico

(ou, como aqui, um sinal constante, caso particular de período 1) é amostrado em uma janela com duração múltipla inteira de seu período, sua TFD apresenta energia apenas nos múltiplos inteiros da frequência fundamental, sem qualquer espalhamento espectral (*leakage*).

Fase. Como $X[1] = X[2] = X[3] = 0$ (números complexos nulos), a função `angle()` do Octave retorna fase 0 por convenção para esses pontos, o que explica os quatro marcadores de fase alinhados em 0 rad na Figura 1 (à direita). Já a fase da TTFD contínua varia de forma linear por trechos entre os zeros do núcleo de Dirichlet, com saltos de π característicos de uma função real que muda de sinal – comportamento típico de pulsos retangulares e consistente com a teoria de filtros FIR de fase linear discutida por Oppenheim e Willsky (OPPENHEIM; WILLSKY; NAWAB, 1996).

3.3 Parte III: Alta Densidade vs. Alta Resolução Espectral

O Código 3 calcula a TFD de $x[n] = \cos(0,48\pi n) + \cos(0,52\pi n)$ para dois comprimentos de janela distintos, $N = 11$ e $N = 101$.

```

1  % Parte III - Alta densidade vs alta resolucao espectral
2  % x[n] = cos(0.48*pi*n) + cos(0.52*pi*n)
3  % a) N = 11 amostras (0 <= n <= 10) -> baixa resolucao
4  % b) N = 101 amostras (0 <= n <= 100) -> alta resolucao
5
6  clear; clc; close all;
7  set(0, 'defaultaxesfontname', 'Arial');
8  set(0, 'defaultaxesfontsize', 11);
9
10 % a)
11 n_a = 0:10;
12 x_a = cos(0.48*pi*n_a) + cos(0.52*pi*n_a);
13 Na = length(n_a);
14 Xa = fft(x_a, Na);
15 fa = (0:Na-1)/Na;           % frequencia normalizada, ciclos
    /amostra
16
17 % b)
18 n_b = 0:100;
19 x_b = cos(0.48*pi*n_b) + cos(0.52*pi*n_b);
20 Nb = length(n_b);
21 Xb = fft(x_b, Nb);
22 fb = (0:Nb-1)/Nb;
23
24 figure('Position', [100 100 900 400]);

```

```

25
26 subplot(1,2,1);
27 stem(fa, abs(Xa), 'filled');
28 xlabel('Frequência normalizada f (ciclos/amostra)');
29 ylabel('|X[k]|');
30 title('TFD - N = 11 amostras (baixa resolução)');
31 xlim([0.15 0.35]); grid on;
32
33 subplot(1,2,2);
34 stem(fb, abs(Xb), 'filled');
35 xlabel('Frequência normalizada f (ciclos/amostra)');
36 ylabel('|X[k]|');
37 title('TFD - N = 101 amostras (alta resolução)');
38 xlim([0.15 0.35]); grid on;
39
40 drawnow;
41 frame = getframe(gcf);
42 imwrite(frame.cdata, '../figuras/parte3.png');
43
44 disp('Parte III concluída. Figura salva.');
```

Código 3.3 – Script para a Parte III.

A resolução espectral de uma TFD de N pontos – isto é, a menor separação em frequência normalizada que permite distinguir duas componentes espectrais distintas – é aproximadamente:

$$\Delta f \approx \frac{1}{N} \quad (4)$$

As duas senóides do sinal possuem frequências normalizadas $f_1 = 0,24$ e $f_2 = 0,26$ (a partir de $\omega = 0,48\pi$ e $\omega = 0,52\pi$, com $f = \omega/2\pi$), separadas por $\Delta f_{\text{sinal}} = 0,02$ ciclos/amostra.

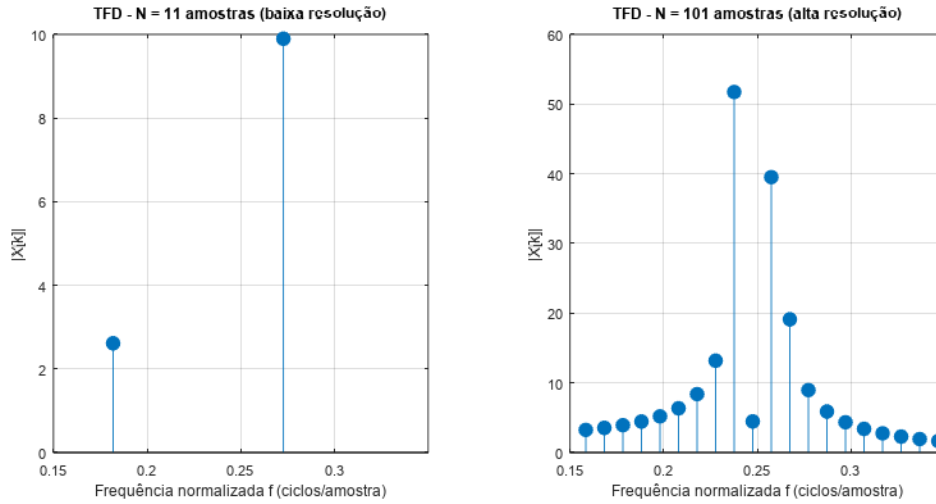


Figura 2 – TFD de $x[n] = \cos(0,48\pi n) + \cos(0,52\pi n)$ para $N = 11$ (baixa resolução, esquerda) e $N = 101$ (alta resolução, direita).

Caso $N = 11$ (baixa resolução). Pela equação (4), $\Delta f \approx 1/11 \approx 0,091$, valor mais de quatro vezes maior do que a separação real entre as duas senoides (0,02). Como consequência, o painel esquerdo da Figura 2 mostra um único lóbulo dominante, próximo de $f \approx 0,273$ (o *bin* mais próximo de k/N que captura a energia combinada das duas senoides), sem qualquer indício visual de que o sinal é, na verdade, composto por duas componentes distintas. As duas frequências são indistinguíveis: diz-se que a TFD, neste caso, não tem resolução suficiente para separá-las, embora os $N = 11$ pontos calculados já representem o espectro completo dessa janela (não há, portanto, falta de *densidade* de pontos – a limitação é estrutural, decorrente da curta duração da observação).

Caso $N = 101$ (alta resolução). Com $N = 101$ amostras, $\Delta f \approx 1/101 \approx 0,0099$, menor que a separação real de 0,02 ciclos/amostra entre as senoides. O painel direito da Figura 2 confirma que, neste caso, as duas componentes são plenamente resolvidas: observam-se dois picos distintos e bem definidos, um próximo de $f \approx 0,238$ e outro próximo de $f \approx 0,257$, com um vale claro entre eles. Os múltiplos lóbulos secundários visíveis entre os dois picos principais são consequência da convolução do espectro com o núcleo de Dirichlet associado à janela retangular de 101 amostras – o mesmo fenômeno de espalhamento espectral discutido na Parte II, porém aqui com lóbulos suficientemente estreitos para não mascarar os dois picos de interesse.

Densidade vs. resolução – distinção conceitual. É importante destacar a diferença entre *aumentar a densidade espectral* (calcular a TFD em mais pontos de frequência através de *zero-padding*, sem alterar a duração real do sinal observado) e *aumentar a resolução espectral* (coletar mais amostras reais do sinal, aumentando sua duração de observação), distinção amplamente discutida por Ingle e Proakis (INGLE; PROAKIS, 2017). No experimento realizado, a transição de $N = 11$ para $N = 101$ corresponde ao segundo caso: foram coletadas 101 amostras genuínas do sinal (não apenas zeros adicionados), o que efetivamente aumenta a resolução,

conforme previsto pela equação (4). Se, em vez disso, tivéssemos aplicado *zero-padding* às 11 amostras originais para obter uma TFD de 101 pontos, o resultado seria uma curva mais densamente amostrada da *mesma* TTFD subjacente – ainda exibindo um único lóbulo largo, pois a informação adicional sobre as duas frequências simplesmente não está presente em apenas 11 amostras reais do sinal.

3.4 Parte IV: Tempo de Execução da FFT

O Código 4 mede o tempo médio de execução da função `fft` para cada N inteiro no intervalo $2 \leq N \leq 2048$, repetindo o cálculo 50 vezes por valor de N e utilizando `clock/etime` para a medição de tempo, conforme solicitado no roteiro.

```
1  % Parte IV - Tempo de execucao da fft para 2 <= N <= 2048
2  % Uso de clock() e etime() conforme solicitado no roteiro
3
4  clear; clc; close all;
5  set(0, 'defaultaxesfontname', 'Arial');
6  set(0, 'defaultaxesfontsize', 11);
7
8  Nvals = 2:2048;
9  reps = 50;
10 tempos = zeros(1, length(Nvals));
11
12 for idx = 1:length(Nvals)
13     N = Nvals(idx);
14     x = rand(1, N);
15     t1 = clock();
16     for r = 1:reps
17         X = fft(x);
18     end
19     t2 = clock();
20     tempos(idx) = etime(t2, t1) / reps;
21 end
22
23 ehprimo = isprime(Nvals);
24
25 figure('Position', [100 100 900 500]);
26 plot(Nvals(~ehprimo), tempos(~ehprimo), '.b', 'MarkerSize',
27     4); hold on;
28 plot(Nvals(ehprimo), tempos(ehprimo), '.r', 'MarkerSize', 6);
29 xlabel('N'); ylabel('Tempo médio de execução (s)');
```

```

29 title('Tempo de execução da fft para 2 \leq N \leq 2048');
30 legend('N composto', 'N primo', 'Location', 'northwest');
31 grid on;
32
33 drawnow;
34 frame = getframe(gcf);
35 imwrite(frame.cdata, '../figuras/parte4.png');
36
37 fprintf('Tempo medio (N primo): %.6e s\n', mean(tempos(
    ehprimo)));
38 fprintf('Tempo medio (N composto): %.6e s\n', mean(tempos(~
    ehprimo)));
39 fprintf('Tempo medio em N=2048 (potencia de 2): %.6e s\n',
    tempos(end));
40 [~, idxmax] = max(tempos);
41 fprintf('Maior tempo observado: N=%d (primo? %d), t=%.6e s\n',
    Nvals(idxmax), ehprimo(idxmax), tempos(idxmax));
42 disp('Parte IV concluida. Figura salva.');
```

Código 3.4 – Script para a Parte IV.

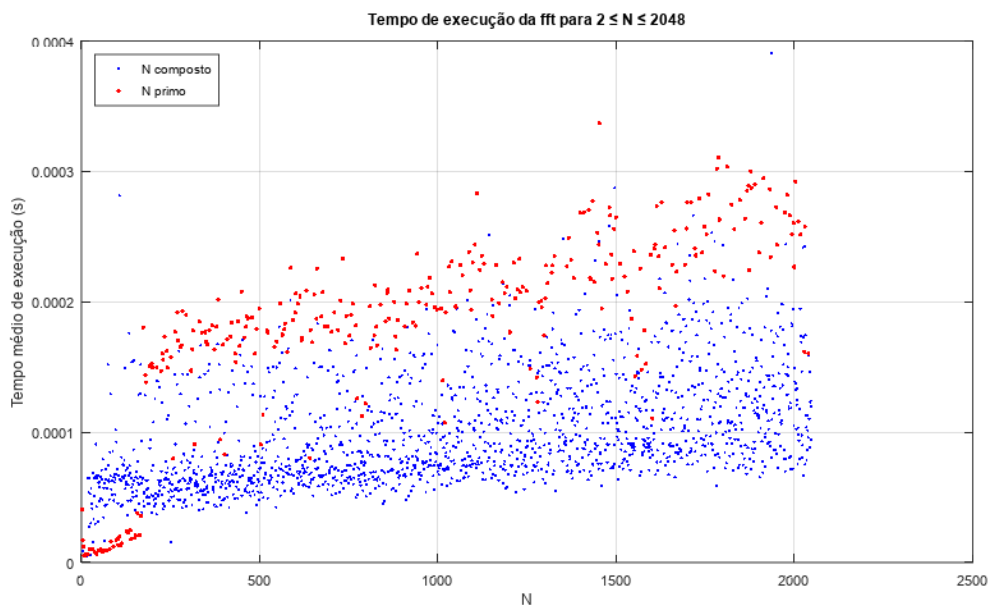


Figura 3 – Tempo médio de execução da fft para $2 \leq N \leq 2048$, separado em N primo (vermelho) e N composto (azul).

Tendência geral. A Figura 3 mostra que o tempo de execução cresce, em média, com N , comportamento consistente com a complexidade assintótica $O(N \log N)$ do algoritmo FFT –

ainda que, para os valores de N aqui avaliados (até 2048), o crescimento absoluto seja modesto, da ordem de microssegundos a décimos de milissegundo.

Primos vs. compostos. A separação por cor evidencia claramente que os valores primos de N (vermelho) apresentam, de forma sistemática, tempos de execução superiores aos valores compostos de mesma ordem de grandeza (azul). Numericamente, o tempo médio para N primo foi de $1,80 \times 10^{-4}$ s, contra $9,90 \times 10^{-5}$ s para N composto – uma razão de aproximadamente $1,8\times$. Essa diferença é explicada pela própria estrutura do algoritmo FFT: quando N é altamente fatorável (em particular potências de 2, como $N = 2048 = 2^{11}$, cujo tempo médio medido foi $1,19 \times 10^{-4}$ s), o algoritmo decompõe recursivamente o problema em subproblemas menores via a estratégia *dividir para conquistar* de Cooley-Tukey, atingindo a complexidade ótima $O(N \log N)$. Quando N é primo, essa decomposição recursiva não é possível, e a biblioteca FFTW (utilizada internamente pelo Octave) recorre a algoritmos alternativos – como o algoritmo de Bluestein, que reescreve a TFD como uma convolução e a calcula via FFTs de tamanho composto maior (tipicamente potência de 2) – que, embora ainda assintoticamente $O(N \log N)$, possuem fator constante de overhead consideravelmente maior, conforme discutido por Proakis e Manolakis (PROAKIS; MANOLAKIS, 2021). Esta razão de aproximadamente $1,8\times$ mostrou-se estável ao se repetir o experimento completo em execuções independentes do Código 4, confirmando que se trata de uma tendência sistemática e não de um artefato de uma única medição.

Valor atípico. O maior tempo isolado observado neste experimento ocorreu em $N = 1936$ (composto, pois $1936 = 2^4 \times 11^2$), com $t \approx 3,90 \times 10^{-4}$ s – um valor bem acima da tendência geral dos compostos vizinhos. Ao repetir a execução completa do Código 4 diversas vezes, observou-se que $N = 1936$ reaparece como o maior tempo isolado na maioria das repetições, o que indica que essa lentidão *não* é mero ruído de medição, mas sim, em boa parte, uma propriedade genuína do algoritmo para esse N específico: embora 1936 seja composto, seu único fator primo além de 2 é 11 (elevado ao quadrado), um fator que a biblioteca FFTW não decompõe de forma tão eficiente quanto os fatores pequenos e mais comuns (2, 3, 5, 7) presentes nos demais compostos do intervalo, aproximando seu desempenho do observado para valores primos. Em ao menos uma das repetições, no entanto, o maior tempo isolado recaiu sobre um valor primo ($N = 1811$) em vez de $N = 1936$, evidenciando que o ruído inerente à medição de tempo de parede (*wall-clock time*) via `clock/etime` – decorrente de interrupções do sistema operacional, efeitos de cache da CPU e variações momentâneas de carga da máquina – também contribui para alguma variabilidade na identidade exata do valor mais lento entre execuções. Isso reforça a importância de repetir cada medição múltiplas vezes (no experimento, 50 repetições por valor de N , além de múltiplas execuções completas do script para esta validação) e analisar tendências médias e reproduzíveis, em vez de valores isolados de uma única execução, ao se avaliar desempenho computacional experimentalmente.

4 CONCLUSÃO

Este laboratório consolidou o estudo da Transformada de Fourier Discreta (TFD) em suas dimensões matricial, espectral e computacional. Através das quatro atividades realizadas, foi possível:

- Implementar e validar as equações vetoriais de análise e síntese da TFD por meio da matriz W_N , identificando suas propriedades de simetria e ortogonalidade, e confirmar numericamente que a formulação matricial é equivalente, a menos de erro de arredondamento de ponto flutuante, à função nativa `fft` do Octave.
- Verificar experimentalmente que a TFD de um sinal de duração N corresponde exatamente às amostras da TFTD nesse sinal nas frequências discretas $\omega_k = 2\pi k/N$, observando o caso particular – e didaticamente revelador – de um pulso retangular cuja TFD de 4 pontos concentra toda a energia na componente DC.
- Distinguir, de forma experimental, os conceitos de densidade espectral e resolução espectral, evidenciando que apenas o aumento genuíno do número de amostras observadas (e não meramente o número de pontos de frequência calculados) é capaz de separar duas componentes de frequência próximas.
- Avaliar empiricamente o tempo de execução da FFT em função de N , confirmando que valores primos de N resultam em tempos de execução sistematicamente maiores do que valores compostos, em concordância com a teoria dos algoritmos de fatoração utilizados pela biblioteca FFTW.

O Octave (EATON et al., 2025) demonstrou-se, novamente, uma plataforma adequada tanto para a implementação numérica direta das equações teóricas quanto para a investigação experimental do desempenho computacional de algoritmos de processamento de sinais. Os resultados obtidos estão em plena concordância com a teoria apresentada por Proakis e Manolakis (PROAKIS; MANOLAKIS, 2021), Oppenheim e Willsky (OPPENHEIM; WILLISKY; NAWAB, 1996) e Ingle e Proakis (INGLE; PROAKIS, 2017).

REFERÊNCIAS

EATON, J. W. et al. **GNU Octave version 10.3.0 Reference Manual**. [S.l.], 2025. Disponível em: <<https://www.gnu.org/software/octave/doc/v10.3.0/>>.

INGLE, V. K.; PROAKIS, J. G. **Digital Signal Processing Using MATLAB**. 4th. ed. [S.l.]: Cengage Learning, 2017.

OPPENHEIM, A. V.; WILLSKY, A. S.; NAWAB, S. H. **Signals and Systems**. 2nd. ed. [S.l.]: Pearson, 1996.

PROAKIS, J. G.; MANOLAKIS, D. K. **Digital Signal Processing: Principles, Algorithms, and Applications**. 5th. ed. [S.l.]: Pearson, 2021.

UFT. **Roteiro de Laboratório 05: Transformada de Fourier Discreta**. 2026. Disciplina de Processamento Digital de Sinais.